Communications Tools How-To: Email and Messaging Channels

This guide explains how to use ragent's built-in communications tools to send and receive email via Gmail, and to post notification messages to external messaging channels (Telegram bots and Discord webhooks) from within an agent session, the TUI, or the HTTP server.

Purpose

ragent ships with two communications tools that let an agent interact with the outside world on behalf of the user:

- **`gmail`** — search, read, draft, and send email through the Gmail REST API v1 using OAuth2 bearer-token authentication.
- **`send_channel_message`** — post text notifications to Telegram chats via a Bot API token, or to Discord channels via incoming webhooks.

Both tools are registered under the `network:send` permission category. They are available in every agent preset by default, but require configuration before they can deliver messages.

Architecture Overview

The communications tools live in two files inside `crates/ragent-tools-extended/src/`:

```
ragent-tools-extended/src/
  gmail.rs       -- GmailTool (OAuth2 + Gmail REST API v1)
  channels.rs    -- SendChannelMessageTool (Telegram + Discord)
```

They are registered in the extended tool registry at `crates/ragent-tools-extended/src/lib.rs`:

```
registry.register(Arc::new(gmail::GmailTool::new()));
registry.register(Arc::new(channels::SendChannelMessageTool));
```

Configuration types live in `crates/ragent-config/src/config.rs`:

- `GmailConfig` — OAuth2 client credentials for the Gmail tool.
- `ChannelsConfig` — master switch and per-channel settings for `send_channel_message`.
- `TelegramChannelConfig` — Telegram bot token, chat ID, and optional API base URL.
- `DiscordChannelConfig` — Discord webhook URL.

Secret Indirection (`env:VAR_NAME`)

Both tools support the `env:VAR_NAME` indirection for any credential field. When a config value is prefixed with `env:`, the tool reads the actual secret from the named environment variable at use time, so secrets never need to live inside the config file. For example:

```
{
  "channels": {
    "telegram": {
      "bot_token": "env:TELEGRAM_BOT_TOKEN",
      "chat_id": "env:TELEGRAM_CHAT_ID"
    }
  }
}
```

The `resolve_secret` helper (in `channels.rs`, re-used by `gmail.rs`) handles this indirection. If the environment variable is unset or empty, the field is treated as missing.

The `gmail` Tool

Overview

The `gmail` tool implements a full Gmail client with seven actions: search, read, draft, send, auth, status, and logout. It uses OAuth2 bearer-token authentication and stores tokens encrypted in the ragent SQLite credential database (the `provider_auth` table), never in `ragent.json`.

Tool name: `gmail` **Permission category:** `network:send` **File:** `crates/ragent-tools-extended/src/gmail.rs`

Authentication Model

The tool supports two ways to become authenticated:

1. **Access token directly** — provide a short-lived OAuth2 access token obtained from the Google OAuth2 Playground:

   ```
   gmail action="auth" access_token="ya29...."
   ```

2. **Refresh token flow** — provide a long-lived refresh token (also from the OAuth2 Playground) along with your OAuth2 client credentials. The tool automatically exchanges the refresh token for short-lived access tokens whenever needed:

   ```
   gmail action="auth" refresh_token="..." client_id="..." client_secret="..."
   ```

The OAuth2 scope must be `https://mail.google.com/`.

Client Credential Resolution When using the refresh-token flow, client credentials are resolved in this precedence order:

1. **Auth-time arguments** — `client_id` and `client_secret` supplied in the `auth` action call (stored encrypted alongside the tokens).
2. **`ragent.json` `gmail.*` fields** — `gmail.client_id` and `gmail.client_secret` in the config file (supports `env:` indirection).
3. **Environment variables** — `GMAIL_CLIENT_ID` and `GMAIL_CLIENT_SECRET`.

Configuration

Add a `gmail` block to `ragent.json`:

```
{
  "gmail": {
    "client_id": "123456789.apps.googleusercontent.com",
    "client_secret": "env:GMAIL_CLIENT_SECRET"
  }
}
```

GmailConfig Fields

Field	Type	Default	Description
`client_id`	string?	`null`	OAuth2 client ID for refresh-token exchange. Supports `env:` indirection. Falls back to `GMAIL_CLIENT_ID` env var.
`client_secret`	string?	`null`	OAuth2 client secret for refresh-token exchange. Supports `env:` indirection. Falls back to `GMAIL_CLIENT_SECRET` env var.
`base_url`	string?	`null`	Optional API endpoint override (defaults to `https://gmail.googleapis.com`). Primarily for testing.

> **Important:** OAuth2 access and refresh tokens are managed by the `gmail` tool itself via the `auth`, `status`, and `logout` actions. They are stored encrypted in the ragent SQLite database — never in `ragent.json`.

Actions

The `gmail` tool accepts a single required `action` parameter that selects the operation. All other parameters depend on the action.

`action="auth"` **— Store OAuth Tokens** Stores OAuth2 credentials in the encrypted ragent credential store. You must supply at least one of `access_token` or `refresh_token`.

Parameters:

Parameter	Type	Required	Description
`action`	string	Yes	Must be `"auth"`.
`access_token`	string	No*	Short-lived OAuth2 access token.
`refresh_token`	string	No*	Long-lived OAuth2 refresh token (enables auto-refresh).
`client_id`	string	No	OAuth2 client ID (stored for refresh).
`client_secret`	string	No	OAuth2 client secret (stored for refresh).

* At least one of `access_token` or `refresh_token` is required.

Example:

```
{
  "action": "auth",
  "refresh_token": "1//0g...",
  "client_id": "123456789.apps.googleusercontent.com",
  "client_secret": "GOCSPX-..."
}
```

Output:

```
Gmail credentials stored (encrypted) in the ragent credential store.
```

Metadata: `{ "authenticated": true, "has_refresh_token": true }`

`action="status"` — Check Credential State Reports whether Gmail credentials are stored and whether an access and/or refresh token is present.

Parameters:

Parameter	Type	Required	Description
`action`	string	Yes	Must be `"status"`.

Example:

```
{ "action": "status" }
```

Output:

```
gmail: authenticated=true, access_token=true, refresh_token=true
```

Metadata: `{ "authenticated": true, "has_access_token": true, "has_refresh_token": true }`

`action="logout"` — Remove Stored Credentials Clears all stored Gmail credentials from the encrypted credential store.

Parameters:

Parameter	Type	Required	Description
`action`	string	Yes	Must be `"logout"`.

Example:

```
{ "action": "logout" }
```

Output:

```
Gmail credentials removed.
```

`action="search"` — List Messages Searches the user's Gmail mailbox using Gmail's native search syntax.

Parameters:

Parameter	Type	Required	Default	Description
`action`	string	Yes	-	Must be `"search"`.
`query`	string	Yes	-	Gmail search query (e.g. `"from:ci@example.com is:unread"`).
`max_results`	integer	No	`10`	Maximum messages to return (clamped to 1-100).

Example — find unread CI emails:

```
{
  "action": "search",
  "query": "from:noreply@github.com is:unread",
  "max_results": 5
}
```

Output:

```
5 message(s) matched (estimate 23).
```

Metadata:

```
{
  "count": 5,
  "result_size_estimate": 23,
  "next_page_token": null,
  "messages": [
    {
      "id": "18c...",
      "thread_id": "18c...",
      "labels": ["UNREAD", "INBOX"],
      "snippet": "CI run #32567192007 failed on main...",
      "headers": {
        "From": "noreply@github.com",
        "To": "user@example.com",
        "Subject": "[ragent] CI run failed",
        "Date": "Fri, 22 Aug 2026 10:25:32 +0000"
      }
    }
  ]
}
```

Gmail search query examples:

Query	Meaning
`is:unread`	All unread messages
`from:alice@example.com`	Messages from a specific sender
`subject:release`	Messages with “release” in the subject
`from:noreply@github.com is:unread`	Unread GitHub notifications
`has:attachment`	Messages with attachments
`label:inbox after:2026/08/01`	Inbox messages after August 1, 2026
`from:boss@company.com newer_than:7d`	Messages from boss in the last 7 days

`action="read"` — Fetch a Single Message Fetches the full message (headers + decoded body) by message ID.

Parameters:

Parameter	Type	Required	Description
`action`	string	Yes	Must be `"read"`.
`id`	string	Yes	Message ID (obtained from `search`).

Example:

```
{
  "action": "read",
  "id": "18c..."
}
```

Output:

```
From: noreply@github.com
Subject: [ragent] CI run failed

CI run #32567192007 failed on main...
```

Metadata includes `id`, `thread_id`, `labels`, `headers`, and `body`. The body is truncated to 4000 characters (`MAX_BODY_SNIPPET`) for display. The full body is included in the metadata `body` field.

`action="draft"` — Create a Draft Creates a Gmail draft (not sent) and returns the draft ID.

Parameters:

Parameter	Type	Required	Description
`action`	string	Yes	Must be `"draft"`.
`to`	string	Yes	Recipient email address.
`subject`	string	No	Subject line (defaults to `"(no subject)"`).
`body`	string	No	Plain-text message body.
`cc`	string	No	Optional Cc header.
`bcc`	string	No	Optional Bcc header.

Example:

```
{
  "action": "draft",
  "to": "team@company.com",
  "subject": "Weekly status report",
  "body": "Hi team,\n\nThis week we completed...\n\nRegards,\nAgent"
}
```

Output:

```
Draft created: r-12345
```

Metadata: `{ "draft_id": "r-12345" }`

`action="send"` — Send an Email Sends an email immediately.

Parameters:

Parameter	Type	Required	Description
`action`	string	Yes	Must be `"send"`.
`to`	string	Yes	Recipient email address.
`subject`	string	No	Subject line (defaults to `"(no subject)"`).
`body`	string	No	Plain-text message body.
`cc`	string	No	Optional Cc header.
`bcc`	string	No	Optional Bcc header.

Example — send a notification:

```
{
  "action": "send",
  "to": "lead@company.com",
  "subject": "Build failed: ragent CI",
  "body": "The CI run on main branch failed. See: https://github.com/thawkins/ragent/actions/runs
}
```

Output:

```
Message sent: 18c...
```

Metadata: `{ "message_id": "18c..." }`

Full Parameters Schema

```
{
  "type": "object",
  "properties": {
    "action": {
      "type": "string",
      "enum": ["search", "read", "draft", "send", "auth", "status", "logout"]
    },
```

```
    "query": { "type": "string" },
    "max_results": { "type": "integer" },
    "id": { "type": "string" },
    "to": { "type": "string" },
    "subject": { "type": "string" },
    "body": { "type": "string" },
    "cc": { "type": "string" },
    "bcc": { "type": "string" },
    "access_token": { "type": "string" },
    "refresh_token": { "type": "string" },
    "client_id": { "type": "string" },
    "client_secret": { "type": "string" }
  },
  "required": ["action"]
}
```

The `send_channel_message` Tool

Overview

The `send_channel_message` tool posts text notifications to externally configured messaging channels. It currently supports two channel kinds:

- **Telegram** — via the Bot API `sendMessage` endpoint.
- **Discord** — via channel webhooks (`POST <webhook_url>`).

Tool name: `send_channel_message` **Permission category:** `network:send` **File:** `crates/ragent-tools-extended/src/`
Max message size: 4096 bytes

Configuration

Channels are configured under the `channels` key in `ragent.json`:

```
{
  "channels": {
    "enabled": true,
    "telegram": {
      "bot_token": "env:TELEGRAM_BOT_TOKEN",
      "chat_id": "env:TELEGRAM_CHAT_ID"
    },
    "discord": {
      "webhook_url": "env:DISCORD_WEBHOOK_URL"
    }
  }
}
```

When `channels.enabled` is `false` (the default), the tool answers `status` and `list` actions but refuses to deliver messages. When no channels are configured at all, it degrades honestly with actionable guidance.

ChannelsConfig Fields

Field	Type	Default	Description
`enabled`	bool	`false`	Master switch. When `false`, `send` is refused but `status`/`list` still work.
`telegram`	`TelegramChannelConfig?`	`null`	Telegram bot channel configuration.
`discord`	`DiscordChannelConfig?`	`null`	Discord webhook channel configuration.

TelegramChannelConfig Fields

Field	Type	Default	Description
`bot_token`	string?	`null`	Bot token from BotFather. Supports `env:` indirection.
`chat_id`	string?	`null`	Chat ID that messages are sent to. Supports `env:` indirection.
`base_url`	string?	`null`	Optional HTTP(S) endpoint override (defaults to `https://api.telegram.org`). Primarily for testing.

DiscordChannelConfig Fields

Field	Type	Default	Description
`webhook_url`	string?	`null`	Full webhook URL (`https://discord.com/api/webhooks/<id>` Supports `env:` indirection.

Actions

The tool accepts an optional `action` parameter (defaults to `"send"`). All other parameters depend on the action.

`action="status"` **— Check Channel Readiness** Reports whether channels are enabled and which channels are configured (with their credential resolution state).

Parameters:

Parameter	Type	Required	Default	Description
`action`	string	No	`"send"`	Set to `"status"`.

Example:

```
{ "action": "status" }
```

Output (when configured):

```
channels.enabled=true, configured=true - [
  {
    "kind": "telegram",
    "configured": true,
    "has_bot_token": true,
    "has_chat_id": true
  },
  {
    "kind": "discord",
    "configured": true,
    "has_webhook_url": true
  }
]
```

Output (when not configured):

```
channels.enabled=false, configured=false - []
```

Metadata includes the full structured payload with `enabled`, `configured`, and a `channels` array.

`action="list"` — List Configured Channels Lists the configured channel kinds and whether they are fully configured (all required credentials resolved).

Parameters:

Parameter	Type	Required	Default	Description
`action`	string	No	`"send"`	Set to `"list"`.

Example:

```
{ "action": "list" }
```

Output:

```
Configured channels: telegram (configured=true), discord (configured=true)
```

`action="send"` — Deliver a Message Sends a text message to one or all configured channels.

Parameters:

Parameter	Type	Required	Default	Description
`action`	string	No	`"send"`	Set to `"send"` (or omit).
`message`	string	Yes	-	Message text to deliver (max 4096 bytes).
`channel`	string	No	`"auto"`	Channel targeting: `"telegram"`, `"discord"`, `"all"`, or omit for auto.

Channel targeting logic:

`channel` value	Behavior
`"auto"` (or omitted)	Sends to Telegram if configured; otherwise sends to Discord.
`"telegram"`	Sends only to Telegram. Fails if Telegram is not fully configured.
`"discord"`	Sends only to Discord. Fails if Discord is not fully configured.
`"all"`	Sends to every configured channel (Telegram + Discord). Reports per-channel results.

Example — send to the first configured channel:

```
{
  "action": "send",
  "message": "CI build #32567192007 passed on main branch."
}
```

Output:

```
[ok] telegram message_id=12345
```

Metadata: `{ "action": "send", "delivered": 1, "failed": [] }`

Example — send to all channels:

```
{
  "action": "send",
  "message": "Release v1.0.48 tagged and pushed.",
  "channel": "all"
}
```

Output:

```
[ok] telegram message_id=12346
[ok] discord webhook delivered
```

Metadata: `{ "action": "send", "delivered": 2, "failed": [] }`

Example — send to Discord only:

```
{
  "action": "send",
  "message": "Alert: CI failed on main.",
  "channel": "discord"
}
```

Full Parameters Schema

```
{
  "type": "object",
  "properties": {
    "action": {
      "type": "string",
      "enum": ["send", "list", "status"]
    },
    "message": {
      "type": "string",
      "description": "Message text to deliver (required for send; max 4096 bytes)"
    },
    "channel": {
      "type": "string",
      "enum": ["telegram", "discord", "all"]
    }
  }
}
```

Provider Feature Matrix

The table below shows which functions/actions are implemented for each provider/channel:

Gmail (Email Provider)

Function / Action	Gmail (OAuth2 REST API)
Search messages	Native
Read message	Native
Create draft	Native
Send email	Native
Store credentials	Native (encrypted)
Check status	Native
Remove credentials	Native

Channel Messaging (Notification Providers)

Function / Action	Telegram (Bot API)	Discord (Webhook)
Send message	Native	Native
List channels	Native	Native
Check status	Native	Native
Custom API endpoint	Native (base_url)	N/A
`env:` secret indirection	Native	Native
Multi-channel broadcast	Via `channel="all"`	Via `channel="all"`

Obtaining Required Tokens

Before the communications tools can send or receive messages, you need to obtain credentials for each provider you intend to use. This section provides detailed, step-by-step instructions for obtaining every token, key, and identifier required by ragent's communications tools.

Token Summary

Provider	Token / Credential	Where to Get It	Lifetime
Gmail	OAuth2 client ID	Google Cloud Console	Per project
Gmail	OAuth2 client secret	Google Cloud Console	Per project
Gmail	OAuth2 refresh token	Google OAuth2 Playground	Long-lived
Gmail	OAuth2 access token	Google OAuth2 Playground	~1 hour
Telegram	Bot token	@BotFather (Telegram)	Until revoked
Telegram	Chat ID	Telegram getUpdates API	Per chat/group
Discord	Webhook URL	Discord channel settings	Until deleted

Obtaining Gmail OAuth2 Credentials

The Gmail tool requires three pieces of information: an OAuth2 **client ID**, an OAuth2 **client secret**, and an OAuth2 **refresh token**. The client ID and client secret identify your Google Cloud application; the refresh token grants ragent long-lived access to your Gmail account.

Prerequisites

- A Google account with Gmail enabled.
- A web browser to complete the OAuth2 consent flow.
- The Google Cloud Console and Google OAuth2 Playground are free to use.

Step 1: Create a Google Cloud Project

1. Open the Google Cloud Console and sign in with the Google account whose Gmail you want ragent to access.
2. Click the project selector dropdown at the top of the page (next to "Google Cloud").
3. Click **New Project** in the top-right of the dialog.

4. Enter a project name (e.g. `ragent-gmail`).
5. (Optional) Select an organization if your Google account is part of a Google Workspace organization. For personal accounts, leave it as "No organization".
6. Click **Create**. Wait a few seconds for the project to be created and selected.

Step 2: Enable the Gmail API

1. In the left sidebar, navigate to **APIs & Services > Library**.
2. In the search bar, type `Gmail API`.
3. Click on **Gmail API** in the results (the one published by Google, with the Gmail icon).
4. Click the **Enable** button. You should see "API enabled" and the Gmail API dashboard.

   Note: If you see "This API is already enabled", you can proceed to the next step.

Step 3: Configure the OAuth Consent Screen

1. In the left sidebar, navigate to **APIs & Services > OAuth consent screen**.
2. For **User type**, select **External** (unless you are using a Google Workspace internal app, in which case select Internal).
3. Click **Create**.
4. Fill in the **App information** form:
   - **App name**: `ragent` (or any name you prefer)
   - **User support email**: your email address
   - **App logo**: (optional, can be skipped)
5. Fill in the **Developer contact information** with your email address.
6. Click **Save and Continue**.
7. On the **Scopes** page, click **Add or Remove Scopes**.
8. Search for `https://mail.google.com/` in the "Manually add scopes" filter box. Check the checkbox next to **Gmail API** with scope `https://mail.google.com/`. This is the full access scope.
9. Click **Update**, then **Save and Continue**.
10. On the **Test users** page, click **Add Users**.
11. Enter the Google account email address that ragent will access (this is your own email, since the app is in "Testing" status).
12. Click **Add**, then **Save and Continue**.
13. Review the summary and click **Back to Dashboard**.

    Important: While the app is in "Testing" status, only the test users you added can authorize it. For personal use this is fine since you added yourself. The app does not need to be published or verified for personal use.

Step 4: Create OAuth2 Client Credentials

1. In the left sidebar, navigate to **APIs & Services > Credentials**.
2. Click **Create Credentials** at the top of the page.
3. Select **OAuth client ID** from the dropdown.
4. For **Application type**, select **Desktop app** (not "Web application" — the Playground uses the desktop flow).
5. For **Name**, enter `ragent-gmail` (or any name you prefer).
6. Click **Create**.
7. A dialog appears showing your **Client ID** and **Client Secret**. Copy both values:
   - **Client ID** looks like: `123456789-abcdefghijklmnop.apps.googleusercontent.com`
   - **Client Secret** looks like: `GOCSPX-AbCdEfGhIjKlMnOpQrStUvWxYz`
8. Click **OK** to close the dialog. You can always find these credentials later under **Credentials > OAuth 2.0 Client IDs**.

   Security: Treat the client secret like a password. Do not commit it to version control. Use the `env:` indirection or environment variables to keep it out of `ragent.json`.

Step 5: Obtain a Refresh Token via the OAuth2 Playground The refresh token is the key credential that lets ragent access your Gmail account long-term. You obtain it by completing a one-time OAuth2 authorization flow through Google's official Playground tool.

1. Open the Google OAuth2 Playground in a new browser tab.
2. Click the **gear icon** (OAuth 2.0 configuration) in the top-right corner of the page.
3. Check the box **Use your own OAuth credentials**.
4. Paste your **Client ID** into the "OAuth Client ID" field.
5. Paste your **Client Secret** into the "OAuth Client secret" field.
6. Close the configuration panel by clicking the gear icon again.
7. In the left panel (Step 1 — Select & authorize APIs), find the "Gmail API v1" section in the scope list. Alternatively, type `https://mail.google.com/` directly into the "Enter your own scopes" text box at the bottom of the list.
8. Check the checkbox next to `https://mail.google.com/` (or click **Authorize APIs** if you entered the scope manually).
9. You will be redirected to Google's consent screen. Sign in with the Google account whose Gmail you want ragent to access (this must be one of the test users you added in Step 3).
10. You may see a warning: "This app isn't verified." This is expected for apps in Testing status. Click **Advanced**, then click **Go to ragent (unsafe)** to proceed.
11. Review the permissions (it will request full Gmail access) and click **Allow** or **Continue**.
12. You will be redirected back to the Playground. In the left panel (Step 2 — Get authorization code), you will see an authorization code in the text box.

13. Click **Exchange authorization code for tokens** (right side of Step 2).
14. The Playground will display:
    - **access_token**: A `ya29...` token (short-lived, ~1 hour).
    - **refresh_token**: A `1//...` token (long-lived).
    - **expiry_date**: When the access token expires.
15. Copy the **refresh_token** value (starts with `1//`). This is the most important credential — it lets ragent automatically obtain new access tokens whenever needed.

    Important: The refresh token does not expire unless you revoke it. Store it securely. If you lose it, you must repeat the authorization flow to obtain a new one.

    Note: If the Playground does not show a refresh token, make sure you checked "Use your own OAuth credentials" and that your OAuth consent screen is configured with the `https://mail.google.com/` scope. Also verify that you added your Google account email as a test user.

Step 6: Store the Credentials in ragent Now that you have the client ID, client secret, and refresh token, store them in ragent:

Option A — Pass everything in the `auth` action:

Tell the agent:

```
Store my Gmail credentials. Use refresh_token "1//0g...",
client_id "123456789.apps.googleusercontent.com", and
client_secret "GOCSPX-...".
```

The agent will call:

```
{
  "action": "auth",
  "refresh_token": "1//0g...",
  "client_id": "123456789.apps.googleusercontent.com",
  "client_secret": "GOCSPX-..."
}
```

All three values are stored encrypted in the ragent credential database. Future Gmail calls will automatically use the refresh token to obtain short-lived access tokens.

Option B — Store client credentials in `ragent.json`, pass only the refresh token at auth time:

Add to `ragent.json`:

```
{
  "gmail": {
    "client_id": "123456789.apps.googleusercontent.com",
    "client_secret": "env:GMAIL_CLIENT_SECRET"
  }
}
```

Export the client secret as an environment variable:

```
export GMAIL_CLIENT_SECRET="GOCSPX-..."
```

Then tell the agent:

`Store my Gmail refresh token "1//0g...".`

The agent will call:

```
{
  "action": "auth",
  "refresh_token": "1//0g..."
}
```

The tool resolves the client ID and secret from `ragent.json` / environment variables during token refresh.

Step 7: Verify Authentication

```
{ "action": "status" }
```

Expected output:

```
gmail: authenticated=true, access_token=true, refresh_token=true
```

Step 8: Revoking Gmail Access (Optional) If you ever need to revoke ragent's access to your Gmail account:

1. Visit myaccount.google.com/permissions.
2. Find `ragent` (or whatever you named the app) in the list of third-party apps with access to your account.
3. Click **Remove Access**.
4. Run `gmail action="logout"` in ragent to clear the stored tokens locally.

Obtaining a Telegram Bot Token and Chat ID

The Telegram channel requires two credentials: a **bot token** (from BotFather) and a **chat ID** (the numeric identifier of the chat or group that messages are sent to).

Prerequisites

- A Telegram account (mobile or desktop app).
- Access to @BotFather (Telegram's official bot management service).

Step 1: Create a Telegram Bot via BotFather

1. Open Telegram and search for **@BotFather** in the contact search bar.
2. Start a chat with @BotFather by clicking **Start**.
3. Send the command `/newbot`.
4. BotFather will ask for a **name** for your bot. This is the display name (e.g. `ragent CI Notifier`). Enter any friendly name.
5. BotFather will ask for a **username**. This must be unique and end with `bot` (e.g. `ragent_ci_notifier_bot`).

6. BotFather will respond with a confirmation message containing your **bot token**. It looks like:

   `123456789:ABCdefGhIjKlMnOpQrStUvWxYz1234567`

   The format is `<bot_id>:<token_string>`.

7. Copy the bot token and store it securely.

   Note: You can regenerate the token at any time by sending `/token` to BotFather and selecting your bot. The old token will be invalidated immediately.

Step 2: Obtain Your Chat ID The chat ID is the numeric identifier of the Telegram chat where the bot will send messages. This can be a private chat with the bot, a group chat, or a channel.

Method A — Private chat (simplest):

1. In Telegram, search for your newly created bot by its username (e.g. `@ragent_ci_notifier_bot`).
2. Start a private chat by clicking **Start**.
3. Send any message to the bot (e.g. "hello"). The bot cannot read messages until you send one first (due to Telegram's privacy model for bots).
4. Open the following URL in your web browser, replacing `<TOKEN>` with your bot token:

   `https://api.telegram.org/bot<TOKEN>/getUpdates`

5. The response will be a JSON object. Look for the `"chat"` object inside the `"result"` array:

```
{
  "ok": true,
  "result": [
    {
      "update_id": 123456789,
      "message": {
        "message_id": 1,
        "from": { "id": 987654321, "is_bot": false, ... },
        "chat": {
          "id": 987654321,
          "first_name": "Your Name",
          "type": "private"
        },
        "date": 1724320123,
        "text": "hello"
      }
    }
  ]
}
```

6. The `"id"` field inside `"chat"` is your chat ID (e.g. `987654321`). For private chats, the chat ID equals your user ID and is a positive number.

Method B — Group chat:

1. Create a new Telegram group or use an existing one.
2. Add your bot to the group:

- Open the group info/settings.
- Click **Add Members**.
- Search for your bot's username and add it.

3. Send any message in the group (e.g. "test message"). The bot must be a member of the group to see messages.
4. Open the following URL in your web browser, replacing `<TOKEN>` with your bot token:

   `https://api.telegram.org/bot<TOKEN>/getUpdates`
5. Find the `"chat"` object with `"type": "group"` or `"type": "supergroup"`:

```
{
  "chat": {
    "id": -100123456789,
    "title": "CI Notifications",
    "type": "supergroup"
  }
}
```

6. The chat ID for groups is a **negative** number (e.g. `-100123456789`). Copy this value exactly, including the minus sign.

   Important: Group chat IDs are negative. The `-100` prefix is part of the ID for supergroups. Do not strip it — ragent sends the chat ID as-is to the Telegram API.

Method C — Channel:

1. Create a Telegram channel (public or private).
2. Add your bot as an **administrator** of the channel:
   - Open the channel info.
   - Click **Administrators** > **Add Admin**.
   - Search for your bot's username and add it as an admin with "Post Messages" permission.
3. Open the getUpdates URL (same as above). The chat ID for channels starts with `-100` (e.g. `-100123456789`), similar to supergroups.

   Note: Bots cannot read messages in channels (they can only post), so `getUpdates` may not show channel messages. If you don't see the channel in the response, use a group or private chat instead, or use the @userinfobot Telegram bot to look up the channel ID.

Step 3: Configure ragent Add the bot token and chat ID to `ragent.json`:

```
{
  "channels": {
    "enabled": true,
    "telegram": {
      "bot_token": "env:TELEGRAM_BOT_TOKEN",
      "chat_id": "env:TELEGRAM_CHAT_ID"
    }
  }
}
```

Export the environment variables:

```
export TELEGRAM_BOT_TOKEN="123456789:ABCdefGhIjKlMnOpQrStUvWxYz1234567"
export TELEGRAM_CHAT_ID="-100123456789"
```

Step 4: Test the Configuration

```
{
  "action": "send",
  "message": "ragent Telegram channel is configured and working."
}
```

Revoking a Telegram Bot Token (Optional) If the bot token is compromised, regenerate it:

1. Send `/token` to @BotFather.
2. Select your bot.
3. BotFather will generate a new token and invalidate the old one.
4. Update your environment variable and restart ragent.

Obtaining a Discord Webhook URL

The Discord channel requires a single credential: a **webhook URL**. Discord webhooks are per-channel and require no authentication token — the URL itself is the credential.

Prerequisites

- A Discord account with access to a server where you have the "Manage Webhooks" permission (typically server administrators or members with the Manage Channels permission).

Step 1: Create a Webhook in Discord

1. Open Discord and navigate to the server where you want notifications to be posted.
2. Select the text channel where messages should appear (or create a new channel, e.g. `#ci-notifications`).
3. Click the **gear icon** (Edit Channel) next to the channel name to open channel settings.
4. In the left sidebar, click **Integrations**.
5. Click **Webhooks**.
6. Click the **New Webhook** button.
7. A new webhook appears with a default name (e.g. "Captain Hook"). Click on it to configure:
   - **Name**: Set a descriptive name (e.g. `ragent CI Notifier`).
   - **Avatar**: (Optional) Upload an avatar image.
   - **Channel**: Verify the target channel is correct.
8. Click **Copy Webhook URL**. The URL looks like:

   `https://discord.com/api/webhooks/1234567890123456780/abcdefghijklmnopqrstuvwxyz0123456789_AB`

   The format is: `https://discord.com/api/webhooks/<webhook_id>/<webhook_token>`.

9. Click **Save Changes**.

   Security: The webhook URL is the only credential needed to post messages. Anyone who has the URL can post to the channel. Treat it like a password and use the `env:` indirection to keep it out of `ragent.json`.

Step 2: Configure ragent Add the webhook URL to `ragent.json`:

```
{
  "channels": {
    "enabled": true,
    "discord": {
      "webhook_url": "env:DISCORD_WEBHOOK_URL"
    }
  }
}
```

Export the environment variable:

```
export DISCORD_WEBHOOK_URL="https://discord.com/api/webhooks/1234567890123456780/abcdefghijklmnop
```

Step 3: Test the Configuration

```
{
  "action": "send",
  "message": "ragent Discord channel is configured and working.",
  "channel": "discord"
}
```

Deleting or Regenerating a Discord Webhook (Optional) If the webhook URL is compromised:

1. Navigate to the channel settings > Integrations > Webhooks.
2. Click on the webhook.
3. Either click **Copy Webhook URL** to get the current URL (it does not change), or click the **Delete** trash icon to remove the webhook entirely, then create a new one.
4. Update your environment variable and restart ragent.

Practical Examples

Example 1: Search and Read Recent Emails

```
Search my Gmail for the latest 5 unread emails from GitHub and
summarize what they say.
```

The agent will:

1. Call `gmail` with `{"action": "search", "query": "from:noreply@github.com is:unread", "max_results": 5}`.
2. For each result, call `gmail` with `{"action": "read", "id": "<message_id>"}`.
3. Summarize the subject and body of each email.

Example 2: Send a Build Notification Email

```
Send an email to lead@company.com with subject "CI Build Passed" and
body "The CI build for ragent v1.0.48 passed all tests."
```

The agent will call:

```
{
  "action": "send",
  "to": "lead@company.com",
  "subject": "CI Build Passed",
  "body": "The CI build for ragent v1.0.48 passed all tests."
}
```

Example 3: Draft an Email for Review

```
Draft an email to team@company.com with subject "Sprint Review" and
body "Please review the attached sprint summary before our meeting."
Don't send it yet, just create a draft.
```

The agent will call `gmail` with `action="draft"` and return the draft ID so the user can review and send it from Gmail.

Example 4: Post a CI Notification to Telegram

```
Send a message to Telegram: "CI build #32567192007 passed on main
branch. All 47 tests green."
```

The agent will call:

```
{
  "action": "send",
  "message": "CI build #32567192007 passed on main branch. All 47 tests green."
}
```

Example 5: Broadcast a Release to All Channels

```
Notify all configured channels that release v1.0.48 has been tagged
and pushed.
```

The agent will call:

```
{
  "action": "send",
  "message": "Release v1.0.48 tagged and pushed to main.",
  "channel": "all"
}
```

Example 6: Check Channel Status Before Sending

```
Check if the messaging channels are configured and ready.
```

The agent will call:

```
{ "action": "status" }
```

And report whether Telegram and Discord are configured and enabled.

Example 7: Combined Email + Channel Workflow

```
Search my inbox for the latest failed CI email, read it to get the
error details, then:
1. Send an email to the team with a summary of the failure.
2. Post a notification to all messaging channels.
```

The agent will: 1. `gmail search` for CI failure emails. 2. `gmail read` the latest one to extract error details. 3. `gmail send` a summary email to the team. 4. `send_channel_message` with `channel="all"` to notify all channels.

Example 8: Cc and Bcc on Outgoing Email

```
Send an email to alice@company.com, cc bob@company.com, and bcc
manager@company.com with subject "Q3 Report" and body "Please find
the Q3 report attached."
```

The agent will call:

```
{
  "action": "send",
  "to": "alice@company.com",
  "cc": "bob@company.com",
  "bcc": "manager@company.com",
  "subject": "Q3 Report",
  "body": "Please find the Q3 report attached."
}
```

HTTP API

Both tools are available via the HTTP server's tool execution endpoint. The server runs on port 9100 by default (`ragent serve --port 9100`).

Gmail via HTTP

```
# Check Gmail auth status
curl -s -X POST http://localhost:9100/tools/gmail \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"action": "status"}'

# Search Gmail
curl -s -X POST http://localhost:9100/tools/gmail \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"action": "search", "query": "is:unread", "max_results": 5}'

# Send an email
```

```
curl -s -X POST http://localhost:9100/tools/gmail \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "action": "send",
    "to": "lead@company.com",
    "subject": "Build passed",
    "body": "All tests green."
  }'
```

Channel Messaging via HTTP

```
# Check channel status
curl -s -X POST http://localhost:9100/tools/send_channel_message \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"action": "status"}'

# Send a notification to all channels
curl -s -X POST http://localhost:9100/tools/send_channel_message \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"action": "send", "message": "Release v1.0.48 pushed.", "channel": "all"}'
```

Troubleshooting

Gmail

“No Gmail access token stored”

You have not authenticated. Run the `auth` action with an access token or a refresh token (plus client credentials):

```
{ "action": "auth", "refresh_token": "...", "client_id": "...", "client_secret": "..." }
```

“finance configuration error: Gmail client secret missing”

You provided a refresh token but the tool cannot find the client secret. Either: - Pass `client_secret` in the `auth` action call. - Set `gmail.client_secret` in `ragent.json`. - Export the `GMAIL_CLIENT_SECRET` environment variable.

“gmail: UNAVAILABLE — credential store unreachable”

The ragent SQLite database cannot be opened (permissions, missing directory). Ensure the ragent data directory (typically `~/.local/share/ragent/`) exists and is writable.

Search returns 0 results

Your Gmail search query may not match any messages. Try broader queries (e.g. `is:unread` instead of `from:specific@sender.com is:unread`). The `result_size_estimate` in the metadata tells you how many total messages Gmail estimates match the query.

Channel Messaging

"Channel messaging is disabled"

The `channels.enabled` flag is `false` (the default). Set it to `true` in `ragent.json`:

```
{ "channels": { "enabled": true, "telegram": { ... } } }
```

"No channels configured"

There is no `channels` block in `ragent.json`, or both `telegram` and `discord` are `null`. Add at least one channel configuration.

"Telegram channel is not fully configured (bot_token and chat_id required)"

You set `channel="telegram"` but either `bot_token` or `chat_id` is missing, or the `env:` variable they reference is unset/empty. Verify the environment variables are exported and non-empty.

"Discord channel is not fully configured (webhook_url required)"

You set `channel="discord"` but `webhook_url` is missing or the `env:` variable it references is unset. Verify the `DISCORD_WEBHOOK_URL` environment variable.

"Message too long: N bytes (max 4096)"

The message exceeds the 4096-byte limit. Split it into multiple shorter messages.

"Telegram send failed (HTTP 401): Unauthorized"

The bot token is invalid or expired. Regenerate it via BotFather.

"Telegram send failed (HTTP 400): Bad Request: chat not found"

The `chat_id` is wrong, or the bot has not been added to the chat. For groups, ensure the bot was added as a member before sending messages.